

Relatório - Exercício Prático 01

Laboratório de Arquitetura de Computadores II

Aluno(a): Arthur De Pinho De Almeida

Matrícula: 888678

Parte 1 - Somadores no Logisim

Foram construídos um meio somador, um somador completo de 1 bit e um somador de 4 bits com propagação de carry, utilizando subcircuitos.

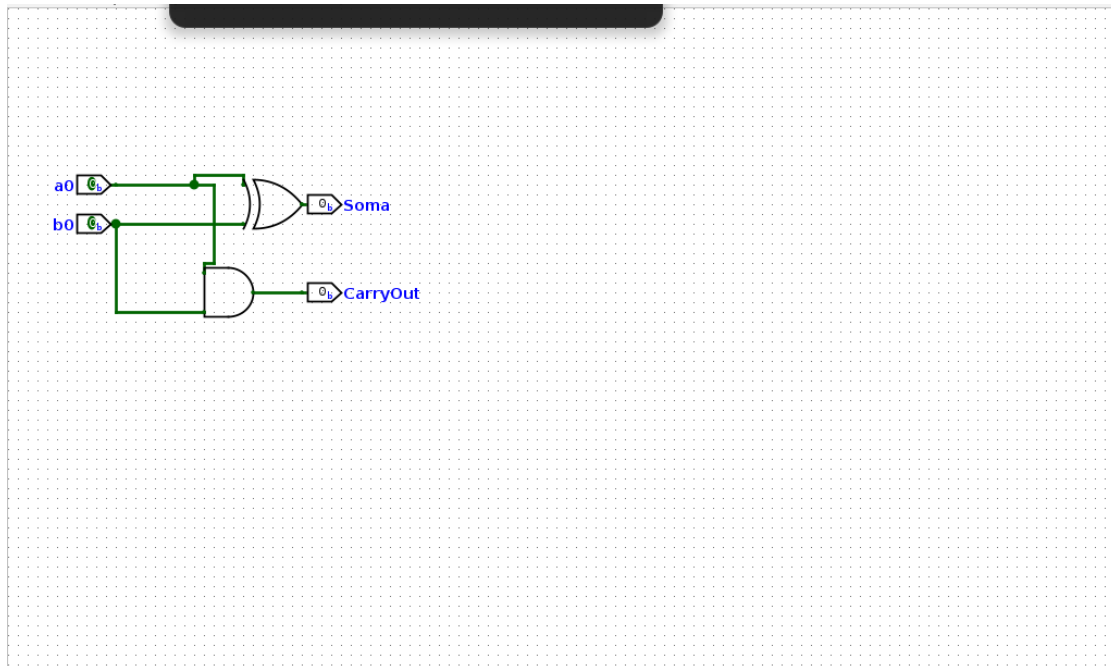


Figura 1 - Meio somador no Logisim.

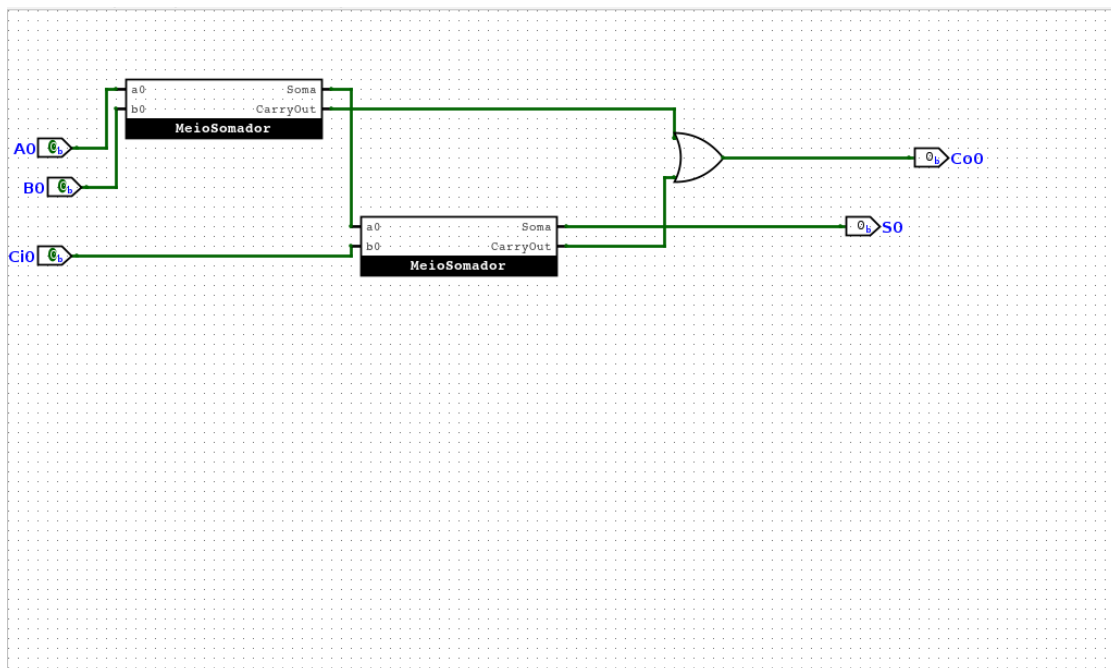


Figura 2 - Somador completo de 1 bit no Logisim.

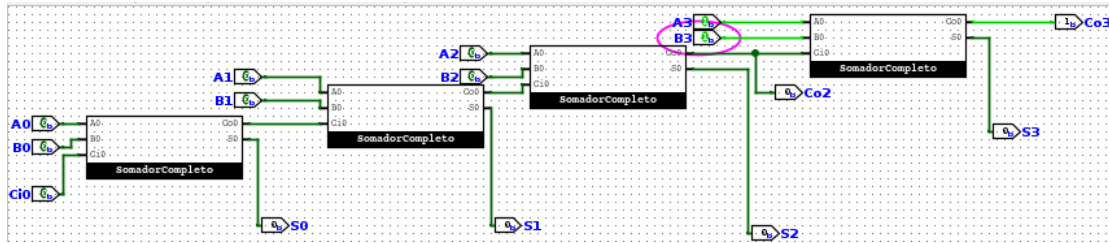


Figura 3 - Soma dos dois primeiros dígitos da matrícula ($8 + 8 = 16$) no somador de 4 bits.

Análise de Tempo

Questão

1. Problema de tempo
2. Soma e carry em 4 bits
3. Somador de 32 bits
4. Frequência em 32 bits
5. Como acelerar

Resposta

O carry de uma posição precisa se propagar para a seguinte. Em um somador ripple-carry, essa dependência cresce linearmente com o número de bits.

Considerando portas de 10 ns e o somador completo construído com dois meio-somadores e uma OR: o carry final tem pior caso de aproximadamente 90 ns; o bit S3 estabiliza em aproximadamente 80 ns. A estimativa considera 30 ns no primeiro estágio e mais 20 ns por estágio de propagação.

Seriam necessários 32 somadores completos encadeados, e o atraso de carry se tornaria significativo.

Pelo mesmo modelo, o carry final tem aproximadamente 650 ns ($30 \text{ ns} + 31 \times 20 \text{ ns}$). Portanto, a frequência máxima é aproximadamente $1/650 \text{ ns} = 1,54 \text{ MHz}$.

Pode-se usar um somador carry look-ahead, carry select ou carry skip para reduzir a dependência sequencial do carry.

Parte 2 - Calculadora de 4 bits

A calculadora recebe duas parcelas de 4 bits, exibe cada parcela e a soma em hexadecimal nos displays de 7 segmentos e possui indicação de overflow.

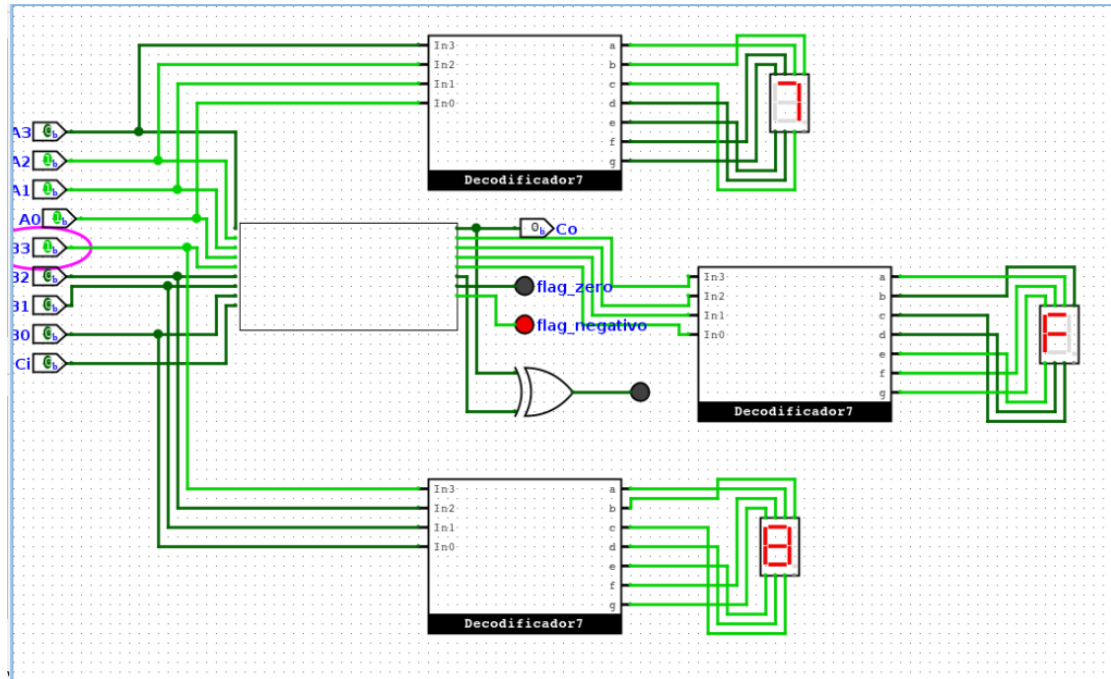


Figura 4 - Circuito principal executando a soma dos finais da matrícula: $7 + 8 = F$.

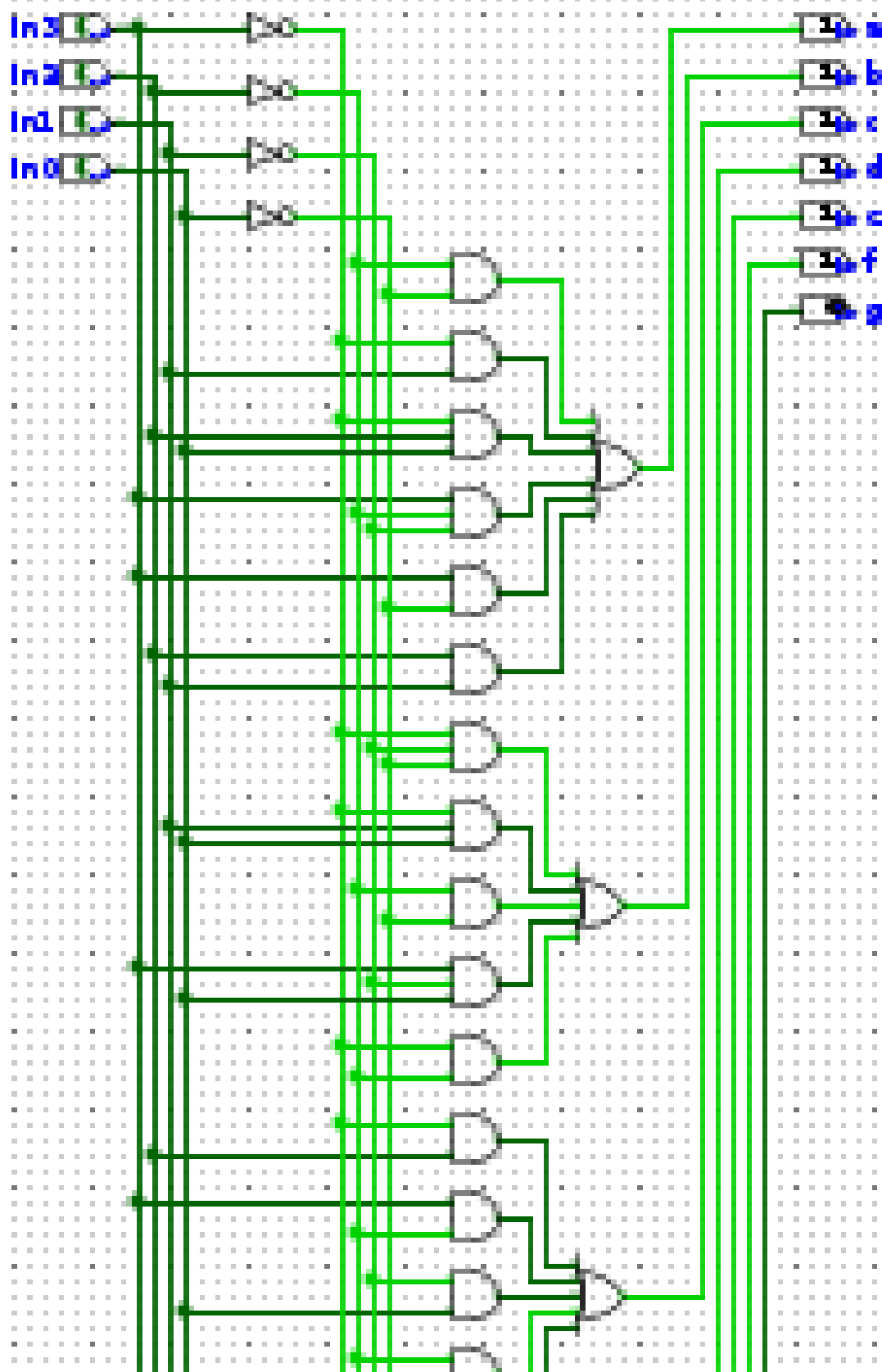


Figura 5 - Estrutura interna do decodificador para display de 7 segmentos.

Parte 3 - Soma com complemento de dois e overflow

O overflow assinado é identificado por $Co2 \text{ XOR } Co3$, isto é, pela diferença entre o carry que entra no bit de sinal e o carry que sai dele.

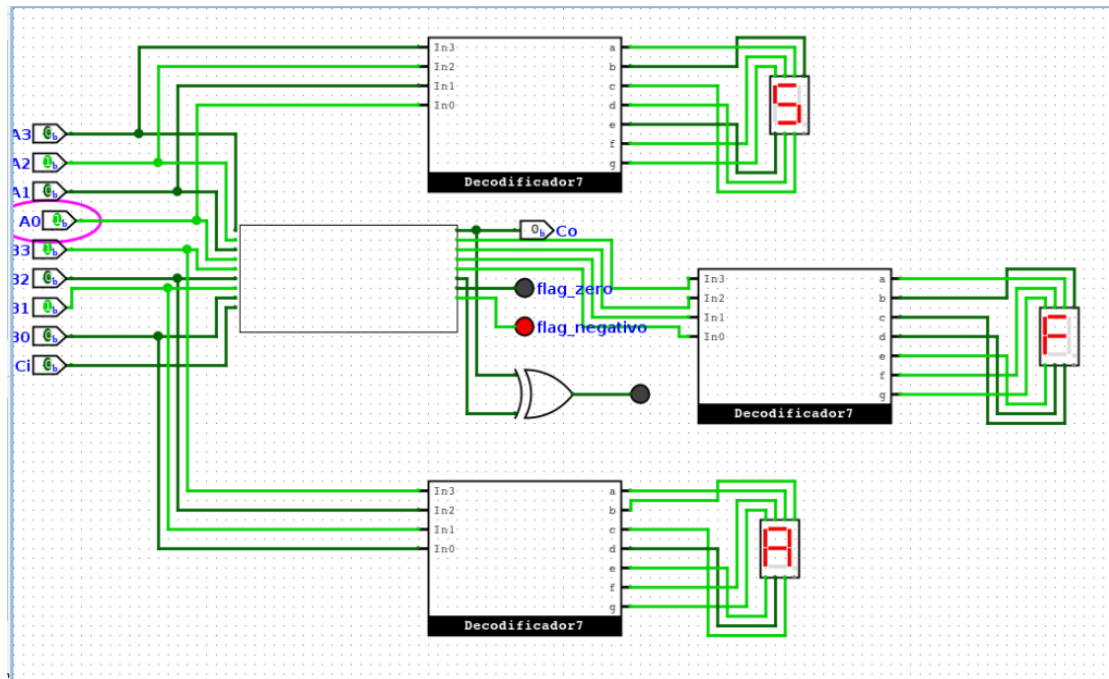


Figura 6 - Operação a: $+5 + (-6)$. Resultado = F (-1).

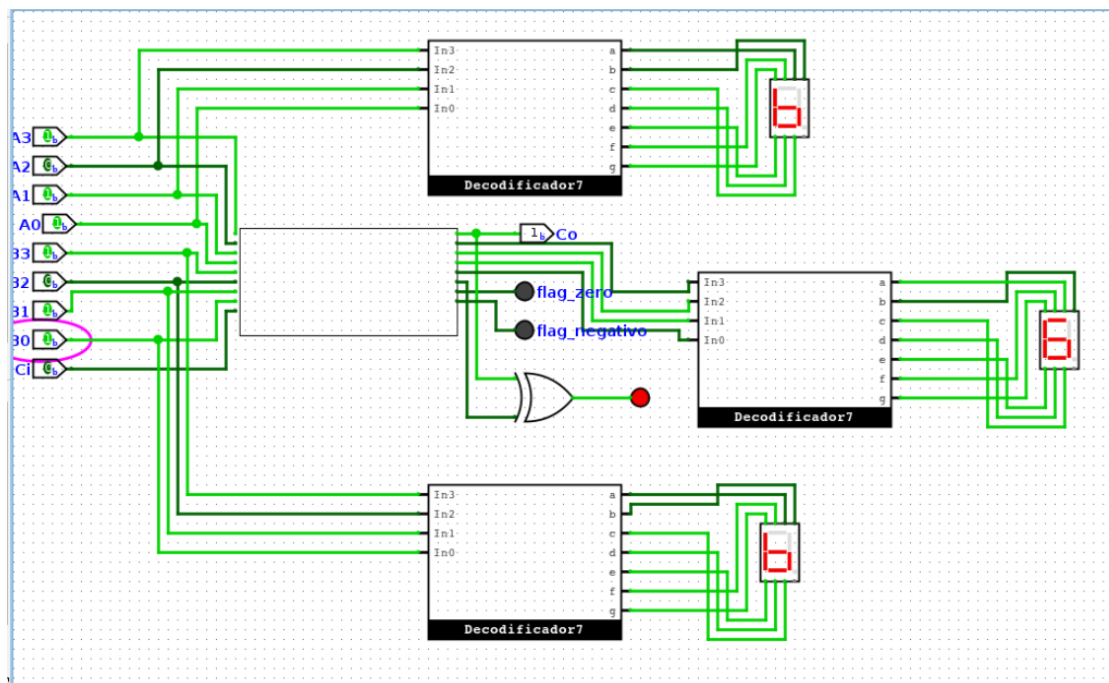


Figura 7 - Operação b: $-5 + (-5)$. Resultado com Overflow ativado (LED aceso).

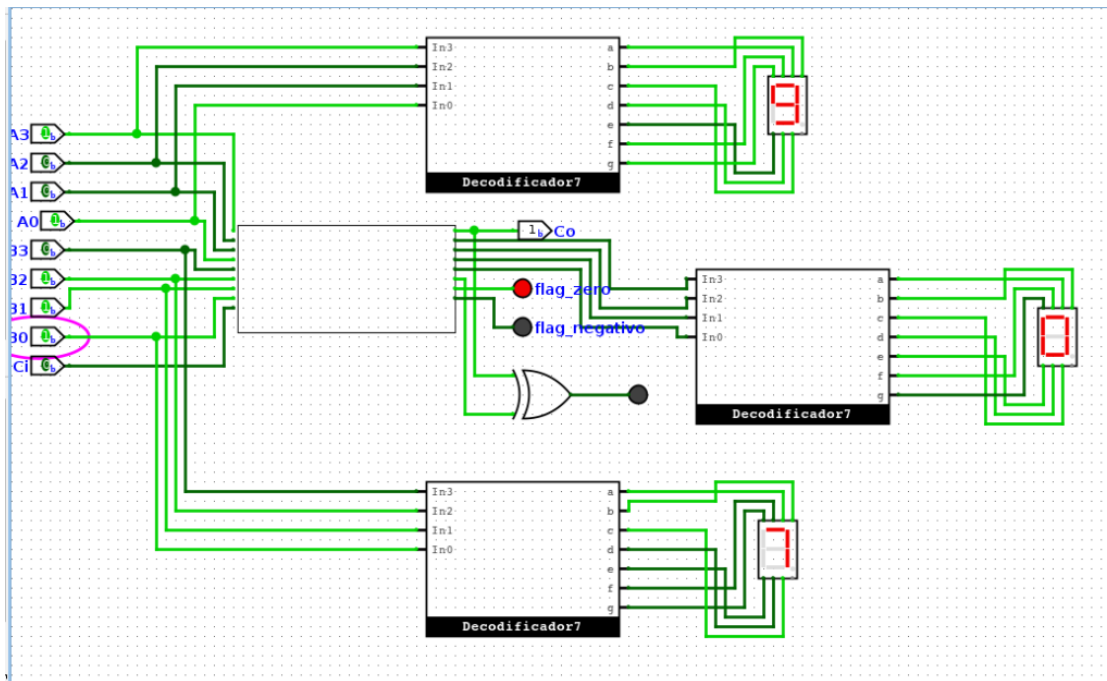


Figura 8 - Operação c: $-7 + (-9)$. Entrada de -9 truncada em 4 bits (vira $+7$).

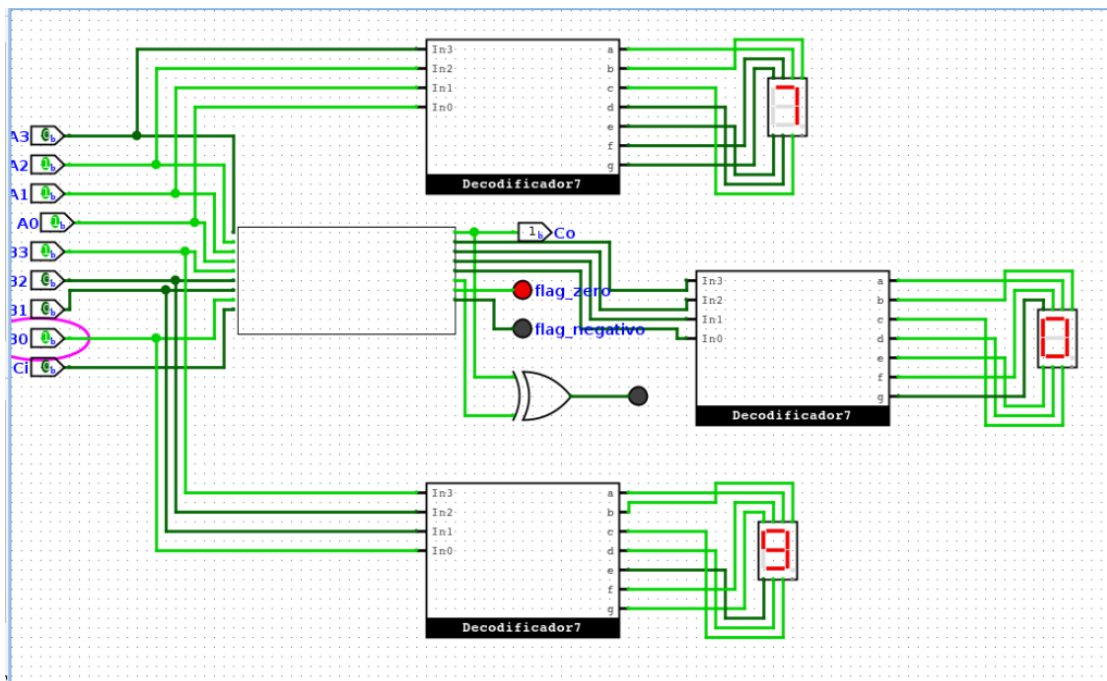


Figura 9 - Operação d: $+7 + +9$. Entrada de $+9$ truncada em 4 bits (vira -7).

Valores exibidos nos displays

Operação	Display A	Display B	Display Soma
a) $+5 + (-6)$	5	A	F

b) $-5 + (-5)$	b	b	6
c) $-7 + (-9)$	9	7	0
d) $+7 + +9$	7	9	0

Resultados e Observação

Questão

a) $+5 + (-6)$

b) $-5 + (-5)$

c) $-7 + (-9)$

d) $+7 + +9$

1) Por que as duas últimas operações deveriam ligar o overflow?

2) Porta lógica utilizada para o overflow

Resposta

Resultado em 4 bits: F (1111), equivalente a -1. Não há overflow assinado.

Resultado em 4 bits: 6 (0110). Há overflow assinado (LED aceso), pois -10 não cabe no intervalo de 4 bits com sinal (-8 a +7).

Em complemento de dois com 4 bits, -9 não é representável. O circuito interpreta -9 como 0111 (+7), calculando $-7 + 7 = 0$. Como são sinais opostos, o hardware não ativa a XOR de overflow.

Em complemento de dois com 4 bits, +9 não é representável. O padrão 1001 é interpretado como -7, calculando $+7 + (-7) = 0$. A XOR permanece em 0.

Matematicamente, $-7 + (-9) = -16$ e $+7 + 9 = +16$, ambos fora do intervalo representável em complemento de dois com 4 bits (-8 a +7), o que por definição constitui overflow. No entanto, como os números ± 9 não possuem representação válida em 4 bits, o hardware os trunca para seus opostos ($-9 \rightarrow +7$ e $+9 \rightarrow -7$). O circuito então processa $-7 + 7 = 0$ e $7 + (-7) = 0$, operações sem overflow real, e por isso o LED de overflow permanece apagado. O overflow ocorreria se os operandos fossem representáveis no intervalo de 4 bits.

XOR (Ou-Exclusivo). Ela gera nível lógico 1 somente quando Co2 e Co3 são diferentes ($\text{Co2 XOR Co3} = 1$), indicando overflow em somas com sinal.

Parte 4 - Flags

A flag Zero é uma porta NOR de quatro entradas: $\text{Zero} = \text{NOT}(\text{S3 OR S2 OR S1 OR S0})$. A flag Negativo é ligada diretamente ao bit mais significativo S3.

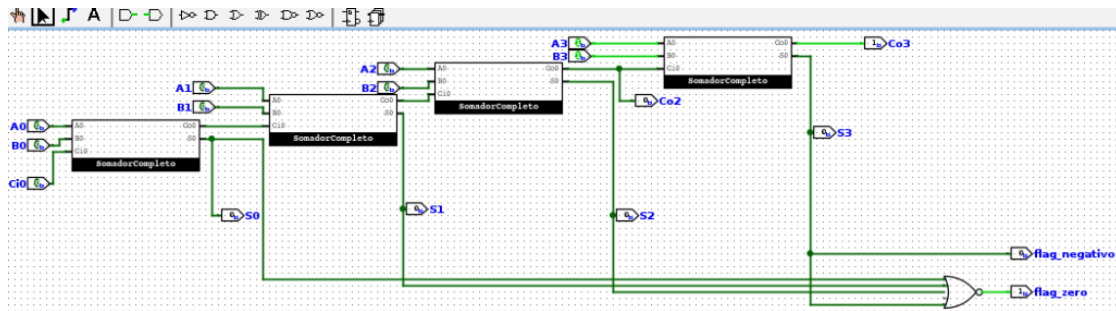


Figura 10 - Implementação interna das flags Zero e Negativo no somador de 4 bits.

Aplicações Práticas: Esses flags são muito usados pelos processadores para ajudar na tomada de decisões (saltos/desvios no código). Por exemplo, o flag Zero serve para o computador saber se dois números são iguais (já que a subtração deles dá zero), e o flag Negativo ajuda a descobrir rapidamente se um número é menor que o outro.

Parte 5 - Somador completo no Tinkercad

O circuito utiliza os CIs 74HC86 (XOR), 74HC08 (AND) e 74HC32 (OR) para implementar um somador completo de 1 bit físico na protoboard.

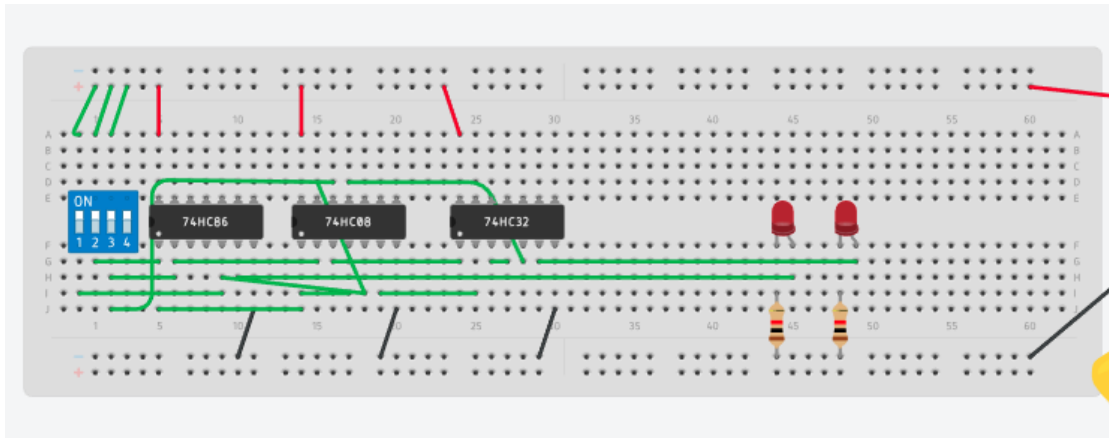


Figura 11 - Somador completo de 1 bit montado no Tinkercad.

Parte 6 - Somador completo no Arduino

O programa em C++ implementa as portas lógicas como funções e aciona os LEDs para indicar a Soma e o Carryout.

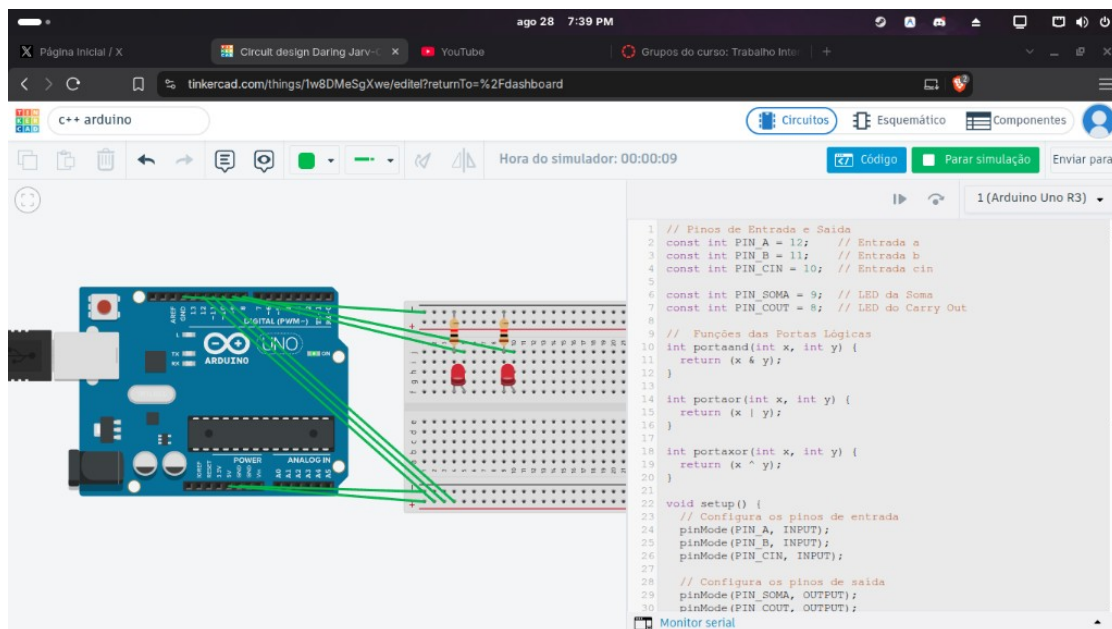


Figura 12 - Simulação do Arduino Uno R3 no Tinkercad.